

Analyzing memory performance of loading Timeseries BSP3

Bachelor Semester Project S3 (Academic Year 2025/26), University of Luxembourg

Alexandra Ene
alexandra.ene.001@student.uni.lu
Université du Luxembourg

Steffen Rothkugel
steffen.rothkugel@uni.lu
Université du Luxembourg

Remus Dobrican
remus.dobrican@ext.uni.lu
Université du Luxembourg

Abstract

Application Programming Interfaces (APIs) play a central role in modern web applications, and their performance is critical due to increasing resource cost, especially in cloud environments. This project addresses the problem of increasing memory usage and performance of two APIs designed to handle time series data. The scenario for this project is defined by a team at an energy company working with time series and running APIs in a cloud environment.

An Apache Cassandra NoSQL open source database, running inside a Docker container, is used for storing and reading time series data with the help of the APIs. The data set was predefined and provided as it would be enough to indicate the existing performance problem. The time series provided form Open Positions (OP), which reflect the OPs of the energy company for a given date, for a certain number of 10 books. These APIs were significantly reduced to a single endpoint, which retrieves the data from the database, which is enough to show the increasing memory problem.

The two APIs developed for using the time series data are designed one with Java 21.0.6, using the Spring 2.7.18 framework, and the other with Python 3.12.6, using the Flask 3.1.2 framework. The monitored system performance and memory usage are compared in order to first show the underlying problem and to understand which framework is faster and more efficient for managing time series.

Finally, a method for manually releasing memory after each call is provided for the Python application.

1 Introduction

In modern web development, APIs have become essential tools that allow applications to communicate and exchange data efficiently. They are widely used in cloud based environments due to the importance of performance and resource management when dealing with growing costs of computational resources.

Companies use cloud services to scale their IT needs using a cost-efficient solution. Big tech companies are using large data centers for providing to the clients services based on their needs, implying a cost schema related to the resources of the machine, the time when computations happen or the traffic. One example is the AWS cloud from Amazon, where clients can acquire EC2 instances with multiple plans, like On-Demand Pricing [1].

2 Time Series Data

A time series database is a database that is specifically optimized for time stamped or time series data. It is made of measurements or events that are tracked, monitored, downsampled, and aggregated over time. They are widely used in domains such as financial marketing, trading, business analytics and energy consumption monitoring.

A NoSQL database is suitable for this, as it does not require a fixed schema, which makes it more flexible and suitable for cases where the data are constantly changing. NoSQL databases are uniquely tailored to their working environments, so they can have a variety of different styles and workflows. However, they do have some characteristics in common. They scale horizontally, which is much more difficult to do with a relational database.

3 Problem Statement

Many applications nowadays work with time series data and managing resources like memory becomes a main concern. As the amount of data increases, memory efficiency becomes an important performance factor, since it affects how well a system can scale and how suitable it is for real world use. Although there are several technologies available for handling time series data, there is limited comparative analysis that focuses specifically on memory usage, where basic operations like loading and unloading data into memory are essential.

In the given scenario, an API written in Python is used to retrieve multiple time series from a Cassandra NoSQL database running in the cloud. The problem is that the RAM memory usage attributed to the API when extracting the data and manipulating it, keeps increasing very fast, without decreasing once the data is not used anymore. This problem is shown in picture [1], where a dashboard in Grafana is used to indicate the never ending and never descending RAM memory usage of the application. This problem leads to reaching the virtual memory limit of the node where the application is running which causes a restart of the application. This restart can happen in the middle of an important operation and can lead to data losses or/and inconsistent data.

The first goal of this study is to compare two commonly used technologies in order to see which gives better results in the context of timeseries data processing. We will compare Python with Flask and Java with Spring and reach our conclusion based on efficiency and performance differences. Based on these analyzes, the final objective is to provide practical insights that can support informed technology selection, especially for applications where memory usage and performance are critical considerations.

4 Reading Memory Consumption for the APIs

Using AI two scripts were created for monitoring the process memory and overall system memory. The purpose is first to show the memory consumption increase after using 10 requests and also the fact that the memory does not decrease even after finishing with the requests. Normally, the memory should be freed by garbage collection in both the case of Python and Java.

Two routes were added to the initial application to monitor the memory of the APIs while running. A simple route just to print

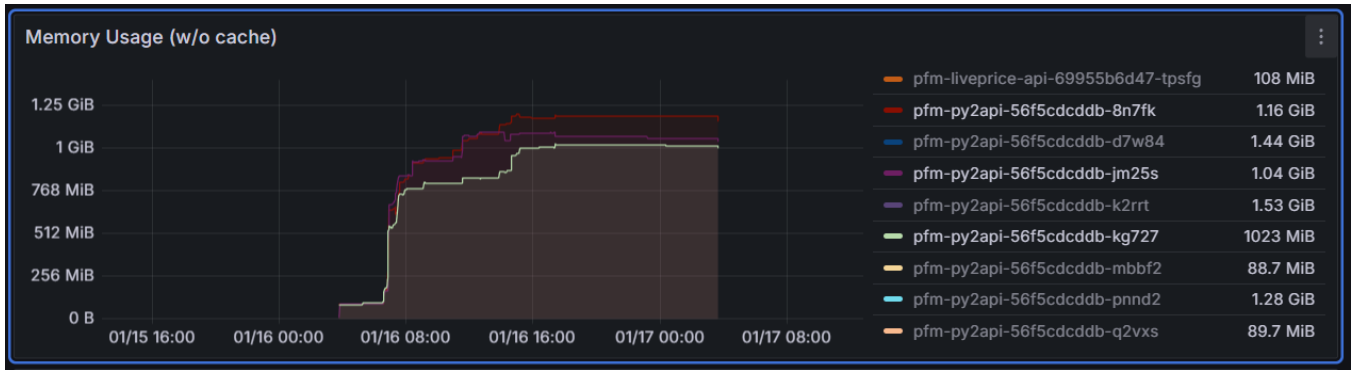


Figure 1: Memory Usage

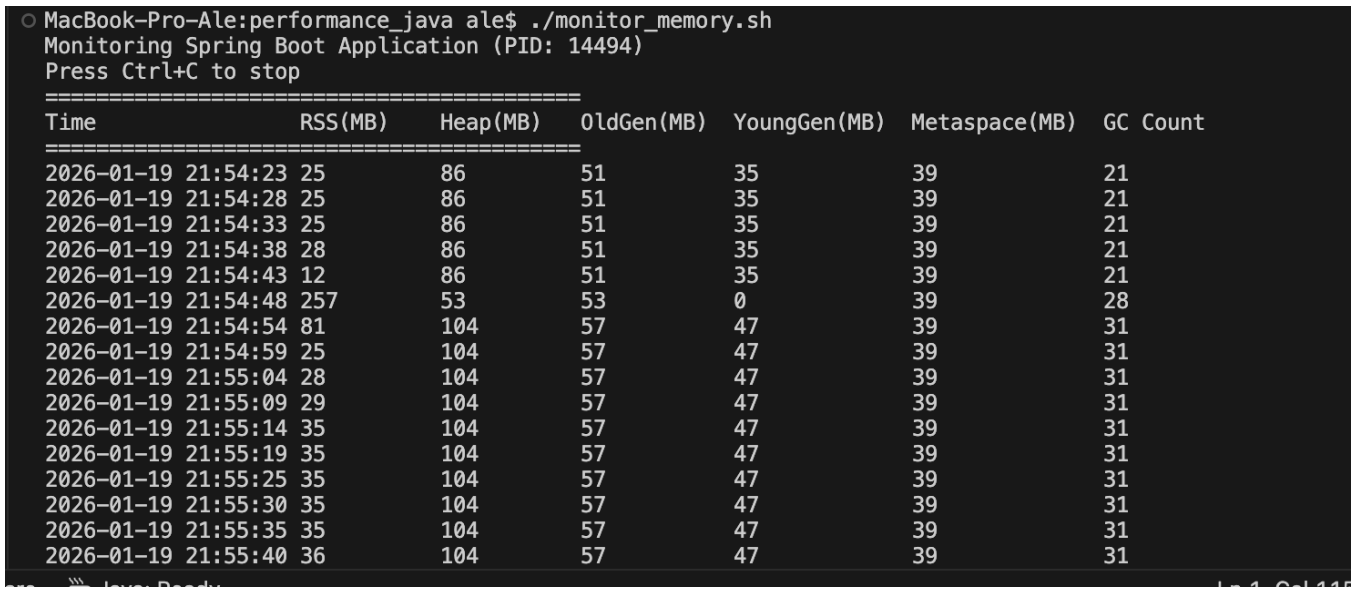


Figure 2: Java Memory Usage

out the RAM memory consumption of the API and another route to print out both the system’s memory consumption together with the API.

5 Memory Release Inside the APIs

A method was created to force the release of the memory, also with the help of AI. The idea behind this was to force the garbage collector to release the memory after this method was implicitly called, at the end of the endpoint which loaded the time series from the data base in the memory. Also, an additional step for cleaning was added, more specifically for cleaning the caches. The code contains Linux specific methods like `libc.malloc_trim()` and `sys._clear_frames()` in the case of the Python application.

6 Experiments

The two APIs and the docker container that contained the Cassandra NoSQL database ran on a MacBook Pro M1 machine, with 8 GB of RAM.

The initial results for running the Python application 10 times an average of 23.97 MB of RAM consumed on the machine without calling the problematic endpoint. Calling the problematic endpoint, without any improvement, took initially 85 MB and went up to 155 MB after 10 endpoint calls as indicated in [4]. These calls were done via the following link for the Python running application http://localhost:8126/d2d_new_2/power?books=LU_STC,DE_PFM,LU_LIG,DE_FLX,DE_PXY,LU_PXY,DE_STC,LU_PFM&as_of_op_old=2025-08-28&as_of_op_new=2025-08-29&ts_start=2025-10-01T00:00:00.000+02:00&ts_end=2030-01-01T00:00:00.000+01:00&aggr=month.

In the second run [3] we have 10 calls with the memory cleanups and the garbage collector (GC) calls, i.e. the `aggressive_memory_cleanup`

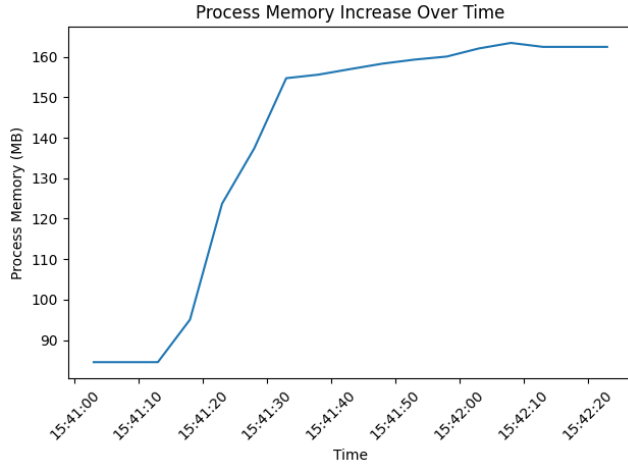


Figure 3: Memory increase

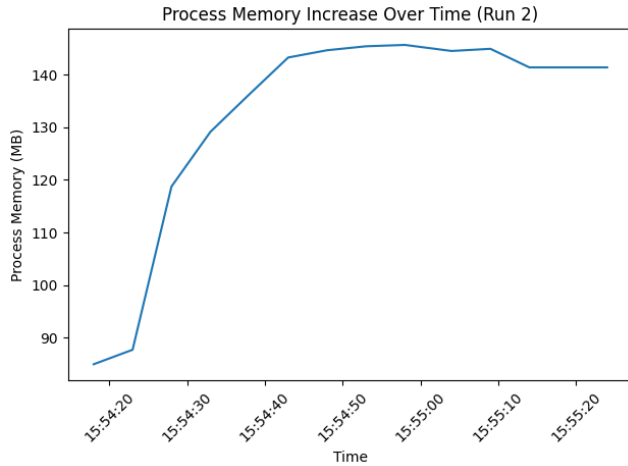


Figure 4: Memory increase 2nd run

method in the Python code. In the warm-up phase 85 MB to 115 MB we had a very fast initial increase, consistent with the Python startup, cache initialization and data loading. Then, in the growth phase, from 115 MB to 145 MB, a smoother, slower increase compared to the first run. Then, in the stabilization there was a slight drop, from 145 to 141 MB, indicating GC activity reclaiming memory, ending in a stable steady state, which is a good sign (no continuous leak).

For the Java-written API, the memory monitoring included the RSS and the Heap memory analysis. These 2 are important as the RSS is checking the /container memory while the Heap memory is used for application behavior, including memory leaks in Java objects, GC pressure / latency, throughput degradation and allocation rate. But these are not the focus in this study.

The same initial tests indicated an average consumption of 20 MB of heap RAM after running the application and without calling the problematic endpoint.

As seen in figure [2], both the RSS and the Heap memories are increasing when the problematic endpoint is called. Even though the Heap goes down from 86 MB to 53, due to the Garbage Collector, it then goes up again to 104 and does not go down afterward to the initial memory when the process started. The RSS memory in exchange is freed and goes down back to 35 MB which is closed to 25 MB.

7 Related Work

Running applications on the cloud, or so called cloud-computing, can be summarized by the need of having an infrastructure capable of providing secure, highly available and scalable services. End users do not have to take care of resource allocation, and these services are handled by an DevOps team. Multiple techniques for cost optimization, including pricing, analysis, and strategies for resource allocation for cloud-computing are presented, for example, in [7].

Bad design and bad architecture of an application can lead to memory problems or leaks [4, 5], which in turn lead to other problems like application restart or service outage (crashes/ out of memory (OOM) kills), severe performance degradation (GC thrashing, paging), and SLA/SLO violations.

In [3] development patterns can explain more than 90% of the memory leaks in the studied projects. Weak design combined with development anti-patterns lead often to problems, shows a study on micro-service architecture [2]. Reducing resources for cloud running applications results in cost reduction but has at a price, planning for resources being one of the key points [6].

8 Apache Cassandra Database

Apache Cassandra is a distributed NoSQL open source database management system made specifically to handle large volumes of data across multiple servers, prioritizing high availability and fault tolerance over consistency. Cassandra is a wide column database, meaning that it can store big tables with a lot of columns, even if not all rows use the same columns. This means that Cassandra is flexible, therefore, a really good choice for datasets where some values may be missing. It can run on clusters made up of many machines, even in different data centers, and does not rely on a single central server. Instead, data are replicated between nodes in a decentralized way, allowing fast read and write operations with low latency.

Cassandra is particularly well suited for timeseries data, as it can efficiently store and retrieve large numbers of records that are indexed by time and partition keys. The initial keyspace is called 'local' and contains two tables 'series_op_power' and 'series_pfc_power'. The primary key is composed of a partition key composed of the 'name' and 'as_of' fields, and of a clustering key based on the 'time' column. The 'name' represents the column where the books are stored and the 'as_of' a date when this data is relevant. The 'time' column shows that each time series has hourly data.

8.1 Cassandra in this project

In this project, Cassandra is used as the main database storing and querying power related timeseries data, including hourly price financial curves (HPFCs) and open position (OPs) series. Each dataset

is made of a large number of hourly observations associated with a specific validation date. This means efficient retrieval of time ranges, which is essential for operations such as day to day comparisons between different valuation dates.

Both the Python Flask backend and the Java Spring Boot backend interact with the same Cassandra schema. To ensure that the database environment is reproducible, isolated and independent of the host operating system, Cassandra is deployed using Docker.

Authentication support is enabled through environment variables defining the Cassandra username and password. These credentials are used consistently by both the Python and Java applications when establishing their database sessions.

Within the application code, Cassandra acts as the storage layer that connects the two backend implementations. The Python application uses Cassandra to insert HPFC and OP datasets and to retrieve timeseries data for analysis and comparison. The Java application connects to the same database to validate data availability and perform day to day checks across different dates.

By relying on Cassandra as a shared data store, the project ensures consistency between both implementations and allows meaningful comparisons of behaviour, performance, and memory usage. Cassandra provides a scalable, resilient, and well suited foundation for managing the large, structured timeseries datasets required by the project, while Docker Compose ensures that this database layer remains portable and reproducible across environments.

9 Data Loading Process

To prepare the database environment for the project, a Cassandra instance was run inside a Docker container. After starting Docker Desktop, the list of active containers was checked to make sure that the Cassandra container was running. Once this was confirmed, a connection was opened directly inside the container, which allowed access to the Cassandra command line interface. From there, the database could be managed and configured, and commands could be executed to create and inspect the necessary data structures.

Inside Cassandra, a separate keyspace was made in order to organize all the related data. Within this keyspace, a table was defined to store timeseries data, including information such as the series name, value, time and additional date related fields. The table was designed in such a way that data belonging to the same series and reference date is grouped together and ordered by time, making it suitable for working with time based data.

After the creation of the table, the dataset was loaded into Cassandra. Simple checks were then performed to make sure the data was stored correctly by retrieving a small number of records. This confirmed that the database was ready to be used by both applications in the project for reading and comparing timeseries data.

10 Reading memory in the API

In the Python application, two additional endpoints were created to access the memory used by the API. The simpler one calling the endpoint `/info/memory/simple` is used by the memory reading script `'monitor_memory.py'` to return the actual memory consumption. It takes the RSS memory info from process library provided by the community supporting the Python language. A second endpoint `/info/memory` is also provided to monitor extra parameters like

the used, available or/and freed memory. If the main Python application is running, then the memory measuring endpoint is called in a loop to check each 5 seconds the status.

For the Spring Java application it is much simpler to get the memory consumption. The bash monitoring script `'monitor_memory.sh'` checks the memory consumption with the help of the PID of the process, obtained by a search with `grep` for `'com.enovos.App'`.

11 Python Application

11.1 Flask

Flask is a lightweight web application framework for Python that is commonly used to build web services and application back-ends. It provides the basic tools needed to define HTTP routes, handle requests, and return responses.

Because of its simplicity and flexibility, Flask is well suited for small to medium sized projects or for building APIs that interact with databases or other services.

In this project, Flask is used to expose the backend functionality through HTTP endpoints, allowing other components to communicate with the Cassandra database in a clear and controlled way.

12 Backend Application

The backend part of the project consists in a Python web application developed with the Flask framework. The main purpose of it is to connect the Cassandra database with the parts of the system that send or request timeseries data. The application provides several HTTP endpoints that allow data to be inserted into the database, retrieved when needed, and lightly processed before being returned. It also includes basic monitoring features that make it possible to observe memory usage and performance while handling larger amounts of data. When the application starts running, it reads its configuration settings from environment variables. After loading the information, the application opens a connection to the Cassandra database using the Datastax Cassandra driver. The connection is set up with authentication and basic timeout settings to ensure reliable communication. Once established, this database connection is shared across the application so that all incoming requests can quickly access Cassandra to read or write data.

Once initialised, the Flask web server exposes some HTTP endpoints. One endpoint is used to check if the service is running and simply returns a fixed response. The application also includes two endpoints for memory monitoring: one provides detailed information about memory usage, while the other returns a simplified value showing only the current memory consumption. These endpoints are mainly used for testing and performance analysis during data processing.

The main functionality of the backend is to receive timeseries data and then store it in Cassandra. This is achieved through an endpoint that accepts JSON data sent by clients. The incoming data is converted into a structured format, and timestamps are created from the provided date information. These timestamps are adjusted to a common time standard before being stored in the database. The data is then inserted into Cassandra in grouped operations to improve performance. Basic checks are performed to ensure that the data is valid, and clear error responses are returned if any problems are detected.

Besides inserting data, the application can also read PFC power data from the database using a dedicated endpoint. Based on these, the application queries Cassandra and returns only the relevant part of the time series. The retrieved data is then converted into JSON format and sent back to the client together with the number of records.

The application also includes functionality for comparing OP power data on a day to day basis. A different endpoint is used to retrieve OP data for selected dates over a given time interval. During this, the application monitors memory usage and performs cleanup operations.

In short, the backend serves as the central layer for managing timeseries data. It provides clearly defined endpoints for storing and retrieving PFC and OP power data, applies basic validation rules, and offers visibility into system performance.

13 Java App

13.1 Spring Boot

Spring is a Java framework used for building large and well structured web applications. It provides a wide set of tools to ease development and organise code into clear components in order to manage how these components work together. One of the core ideas behind Spring is dependency injection, which means that objects are created and connected automatically by the framework instead of manually in the code. This makes applications easier to maintain and extend.

Spring Boot is a simplified version of Spring. It reduces configuration effort and allows applications to start quickly with sensible default settings. It automatically sets up a web server and loads required components, handling common tasks such as request routing and configuration management.

13.2 Backend Application

The Java version of the backend is developed using the Spring Boot framework. The App class contains the main method and is marked as the main entry point of the program. When this class is executed, Spring Boot automatically sets up the application environment, starts the web server, and scans the project for controllers and services. As a result, all components are initialised and made ready to run.

An important part of this setup is the application configuration class. This class is responsible for preparing and configuring the objects that the application needs in order to work correctly. It is processed when the application starts, before any requests are handled. The configuration focuses mainly on database connections and logging. It defines how the application connects to the Cassandra database and ensures that these connections can be reused throughout the application. Spring Boot makes it easier to manage dependencies and ensures that all parts of the backend use the same configuration.

The first part of the configuration creates a database connection using settings from the application, such as the database address and login details. This connection can be used by the application whenever it needs to work with a relational database. To make debugging easier, the connection is wrapped with a logging tool that records all SQL queries automatically. The wrapped connection

is set as the default one used by the application, which helps with testing and troubleshooting during development.

The second and most important part of the configuration sets up the connection to the Cassandra database. A Cassandra session is created using the official Java driver and is managed by Spring. This connection handles tasks such as authentication and connection management. The configuration includes the correct datacenter name so that Cassandra can handle requests correctly. Access to the database is secured using a username and password. When the application starts, this connection is created once and then reused throughout the application, allowing all backend components to interact with Cassandra without having to handle the connection setup themselves.

Overall, this configuration class is a key part of the Java backend structure. It hides the technical details of setting up database connections and ensures that all required connections are ready before the application starts handling requests. The relational database setup supports query logging to help with debugging, while the Cassandra connection is used for storing and accessing timeseries data. By keeping these responsibilities in a separate configuration component, the application stays well organised, easier to maintain, and follows common Spring Boot design practices.

14 Comparative Analysis of the Python and Java Implementations

The project includes two separate backend applications that both connect to the same Cassandra database and operate on the same data. Even though both applications are designed to perform the same tasks, they differ in how they are structured, how they run, and how they manage resources.

14.1 Architectural Style and Frameworks

The Python application uses a simple structure. Flask is used to define web endpoints and to connect incoming requests to Python functions. Most of the logic is written directly inside functions, without a strict separation into many layers. The application communicates with Cassandra using the Python Cassandra driver, and Pandas is used to prepare and transform the data before it is stored or returned. This approach is easy to understand and quick to develop. It allows changes to be made rapidly.

The Java application is built using Spring Boot and follows a more structured design. The code is organised into controllers, which handle incoming requests and services, which contain the database access and main logic. Spring is used to manage components, and important elements such as the Cassandra connection. This approach enforces a clear separation between different parts of the application and provides better control over how components are created and managed. Because of this, the Java application follows common enterprise development practices, while the Python version is more flexible and better suited for experimentation and prototyping.

14.2 Dependency Management and Build Systems

The Python application runs as an interpreted program and uses simple tools such as pip to manage its dependencies. It mainly

depends on libraries like Flask, Pandas, and the Cassandra driver. Once these packages are installed, the code can be run directly. This makes the Python version easy to set up and quick to modify, but it also means that differences between environments can appear if versions are not carefully controlled.

The Java application is managed using Maven, where all dependencies and settings are defined in a configuration file. This makes the build process more structured and ensures that the same versions are used every time. The application is compiled into a single executable file that includes the web server. This makes the Java version more predictable and easier to run consistently across systems, although it requires a more complex build process compared to Python.

14.3 Database Integration

In both applications, Cassandra is used as the main database where all timeseries data is stored. In the Python backend, data retrieved from Cassandra is loaded into Pandas DataFrames. This makes it easy to work with timeseries data, apply transformations, merge OP and HPFC datasets, and adjust timestamps. Pandas provides a flexible and convenient way to handle these operations with relatively little code.

In the Java backend, Cassandra is accessed using the Datastax Java Driver. The results returned from the database are mapped into Java objects instead of table structures. This approach is more strict and helps catch errors at compile time, but it also requires more detailed code to handle queries and process results. In addition, the Java application uses multiple threads to process several datasets at the same time, while the Python version mainly processes data step by step unless extra mechanisms for parallel execution are added.

14.4 REST API Behaviour

From a functional point of view, both backends provide similar API endpoints, such as retrieving data or checking whether OP timeseries data exists for different reference dates. The main difference lies in how each framework handles incoming requests.

In Python, Flask endpoints are defined as simple functions, and tasks such as reading request data, validating inputs, and handling errors are written manually inside each function. In contrast, the Java backend uses Spring Boot controllers, which automatically convert request parameters into structured objects, handle JSON responses, and provide support for error handling.

Because of this, the Java implementation offers more structure and stronger checks at development time, while the Python implementation remains more flexible and compact, making it easier to adapt during experimentation.

14.5 Performance and Memory Management

In terms of performance, the Java application manages memory in a structured way and optimises code during execution. This means Java is more predictable and suitable for long running workloads, especially when memory usage needs to be monitored.

Python is easier to develop with but runs in an interpreted environment and has limitations. While libraries like Pandas help improve performance, Python's memory usage is harder to control.

As a result, Python is more flexible for experimentation, while Java offers more stable performance.

14.6 Development Effort and Readability

The Python version of the application is much shorter and easier to read. Flask requires very little setup code, and new features can be added quickly. This makes Python especially suitable for working with timeseries data and numerical processing.

The Java version is more complex and requires more structure, including multiple classes and configuration files. Although this makes the code longer, it also improves clarity and reliability. The stricter structure helps catch many errors earlier, during compilation and makes the application easier to maintain as it grows.

14.7 Stability and Suitability

The Python and Flask implementation is well suited for rapid development, experimentation, and data analysis, where flexibility and speed are important.

In contrast, the Java and Spring Boot implementation follows a more structured approach. It focuses on strong typing and controlled execution, which makes it better suited for production systems where stability, monitoring, and scalability are key requirements.

15 Overall Conclusion

Both implementations successfully solve the same functional problem yet they differ in two fundamentally different backend development philosophies. Python prioritises agility and developer productivity, while Java emphasises structure, robustness, and operational control. This contrast provided valuable insight into how technology selection can influence not only performance and resource usage, but also the maintainability, deployability, and long term sustainability of a system.

In Python, freeing up memory can be done manually by the user, by invoking the garbage collector, thus reducing the risk of out of memory events. This reduces a lot the heavy memory increase and can give the user, combined with careful resource planning, a good way of reducing costs while running cloud-based applications.

16 Future Work

In this study we compared the two frameworks: Python with Flask and Java with Spring. For a more in depth observation, future work could further compare some other frameworks like FastAPI (Python) and Quarkus (Java). Further research could also focus on scalability aspects, such as evaluating system behavior under higher data volumes, increased request rates, or distributed deployments.

References

- [1] Amazon Web Services, Inc. [n.d.]. Amazon EC2 On-Demand Pricing. <https://aws.amazon.com/ec2/pricing/on-demand/>.
- [2] Alberto Avritzer, Andrea Janes, Catia Trubiani, Helena Rodrigues, Yuanfang Cai, Daniel Sadoc Menasché, and Álvaro José Abreu de Oliveira. 2025. Architecture and Performance Anti-patterns Correlation in Microservice Architectures. In *Proceedings of the 2025 IEEE 22nd International Conference on Software Architecture (ICSA 2025)*. IEEE, 60–71. doi:10.1109/ICSA65012.2025.00016

- [3] Jie Chen, Dongjin Yu, and Haiyang Hu. 2023. Towards an Understanding of Memory Leak Patterns: An Empirical Study in Python. *Software Quality Journal* 31, 4 (2023), 1303–1330. doi:10.1007/s11219-023-09641-5
- [4] Yuhuai He, Guanqun Zhang, Zhiting Li, Yong Wang, Zhiqiang Zhang, and Stefano Ermon. 2018. Deep Residual Memory Networks for Language Modeling. *arXiv preprint* (2018). arXiv:1810.00101
- [5] Chang Lou, Cong Chen, Peng Huang, Yingnong Dang, Si Qin, Xincheng Yang, Xukun Li, Qingwei Lin, and Murali Chintalapati. 2022. RESIN: A Holistic Service for Dealing with Memory Leaks in Production Cloud Infrastructure. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI'22)*. USENIX Association, Carlsbad, CA, USA, 109–125.
- [6] Xuan Wang, Liping Zhang, Junjian Zhang, Tao Xu, and Shoushuai Liu. 2023. Performance Evolution Analysis of Microservice System Based on Service Dependency and Structural Complexity. *Journal of Systems and Software* 200 (2023), 111635. doi:10.1007/s10922-023-09726-3
- [7] Xinyu Zhang, Yanfeng Zhou, Yang Liu, Yutao Wang, Peng Xu, Tianyin Zhang, and Hao Chen. 2023. Understanding Memory Leaks in Python: A Large-Scale Empirical Study. *arXiv preprint* (2023). arXiv:2307.12479